

Funktionale Programmierung in F# (4)

Domain Driven Design & Property Based Testing

Göran Kirchner¹

2026-03-18

¹e_kirchnerg@doz.hwr-berlin.de

Programm

- Domain Driven Design (DDD)
- Property Based Testing
- Exkurs: FP + Logic → [Lean4](#)

DDD

↪ Domain Driven Design

–Scott Wlashin: F# for Fun and Profit

Prinzipien

- Verwende die Sprache der Domäne (ubiquitous Language)
- Values und Entities
- der Code ist das Design (kein UML nötig)
- Design mit (algebraischen) Typen
 - Option statt Null
 - DU statt Vererbung
- illegale Konstellationen sollten nicht repräsentierbar sein!

Pause

Are you quite sure that all those bells and whistles, all those wonderful facilities of your so called powerful programming languages, belong to the solution set rather than the problem set?

– Edsger Dijkstra

DDD Übung 1 (Contacts – ex 2)

A Contact has

- a personal name
- an optional email address
- an optional postal address
- Rule: a contact must have an email or a postal address

A Personal Name consists of a first name, middle initial, last name

- Rule: the first name and last name are required
- Rule: the middle initial is optional
- Rule: the first name and last name must not be more than 50 chars
- Rule: the middle initial is exactly 1 char, if present

A postal address consists of a four address fields plus a country

- Rule: An Email Address can be verified or unverified

DDD Übung 2 (Payments – ex 3)

The payment taking system should accept:

- Cash
- Credit cards
- Cheques
- Paypal
- Bitcoin

A payment consists of a:

- payment
- non-negative amount

After designing the types, create functions that will:

- print a payment method
- print a payment, including the amount
- create a new payment from an amount and method

DDD Übung 3 (Refactoring – ex 4)

Much C# code has implicit states that you can recognize by fields called "IsSomething", or nullable date.

This is a sign that states transitions are present but not being modelled properly.

DDD Übung 4 (Shopping Cart – fsm ex 3)

Create types that model an e-commerce shopping cart.

- Rule: "You can't remove an item from an empty cart"!
- Rule: "You can't change a paid cart"!
- Rule: "You can't pay for a cart twice"!

States are:

- Empty
- ActiveCartData
- PaidCartData

Pause

About the use of language: it is impossible to sharpen a pencil with a blunt axe. It is equally vain to try to do it with ten blunt axes instead.

– Edsger Dijkstra

Example Based Tests :)

```
module Test1 =  
  open Implementation1  
  let tests = testList "implementation 1" [  
    test "add 1 3 = 4" {  
      let actual = add 1 3  
      let expected = 4  
      Expect.equal expected actual "" }  
    test "add 2 2 = 4" {  
      let actual = add 2 2  
      let expected = 4  
      Expect.equal expected actual "" } ];;  
runTests expectoConfig Test1.tests
```

```
runTests expectoConfig Test1.tests;;
```

```
Expecto Running...
```

```
[14:51:49 INF] EXPECTO? Running tests... <Expecto>
```

```
[14:51:49 INF] EXPECTO! 2 tests run in 00:00:00.0366264 for implementation 1 -  
val it: int = 0
```

Evil Developer From Hell :(

```
module Implementation1 =  
  let add x y =  
    4
```

```
module Implementation1 =  
  val add: x: 'a -> y: 'b -> int
```

PBT

↪ Property Based Testing

–Scott Wlashin: F# for Fun and Profit

FsCheck

```
let add1 x y = x + y
let add2 x y = x - y
let commutativeProperty f x y =
    let result1 = f x y
    let result2 = f y x
    result1 = result2;;
FsCheck.Check.Quick (commutativeProperty add1)
FsCheck.Check.Quick (commutativeProperty add2)
```

```
FsCheck.Check.Quick (commutativeProperty add1)
FsCheck.Check.Quick (commutativeProperty add2);;
Ok, passed 100 tests.
Falsifiable, after 2 tests (3 shrinks) (StdGen (1966947397, 297601188)):
Original:
1
-2
Shrunk:
0
1
val it: unit = ()
```

FsCheck (Generate)

```
type Temp = F of int | C of float;;
let fGen =
  FsCheck.Gen.choose(32,212)
  |> FsCheck.Gen.map (fun i -> F i);;
let cGen =
  FsCheck.Gen.choose(0,100)
  |> FsCheck.Gen.map (fun i -> C (float i));;
let tempGen =
  FsCheck.Gen.oneof [fGen; cGen]

let test = tempGen |> FsCheck.Gen.sample 0 20
test
```

```
let tempGen =
  FsCheck.Gen.oneof [fGen; cGen]
```

```
let test = tempGen |> FsCheck.Gen.sample 0 20
test;;
```

```
val tempGen: FsCheck.Gen<Temp> = Gen <fun:Bind@88>
```

```
val test: Temp list =
```

```
[F 44; C 67.0; C 75.0; C 98.0; C 88.0; F 199; F 167; F 144; F 112; C 27.0;
  C 17.0; C 37.0; C 27.0; F 160; F 128; F 105; F 73; F 50; F 199; F 176]
```

```
val it: Temp list =
```

FsCheck (Shrink)

```
open FsCheck
let smallerThan81Property x = x < 81
FsCheck.Check.Quick smallerThan81Property

let test1 = FsCheck.Arb.shrink 100 |> Seq.toList
let test2 = FsCheck.Arb.shrink 88 |> Seq.toList
test2
```

Falsifiable, after 85 tests (0 shrinks) (StdGen (1967637707, 297601188)):
Original:

81

```
val smallerThan81Property: x: int -> bool
val test1: int list = [0; 50; 75; 88; 94; 97; 99]
val test2: int list = [0; 44; 66; 77; 83; 86; 87]
val it: int list = [0; 44; 66; 77; 83; 86; 87]
```


Auswahl der Eigenschaften

- Unterschiedlicher Weg, gleiches Ziel
($\text{Map}(f)(\text{Option}(x)) = \text{Option}(f\ x)$)
- Hin und Her (z.B. Reverse einer Liste)
- Invarianten (z.B. Länge einer Liste bei Sortierung)
- Idempotenz (noch einmal bringt nichts mehr)
- Divide et Impera! (teile und herrsche)
- Hard to prove, easy to verify (Primzahlzerlegung)
- Test-Orakel (z.B. einfach aber langsam)

Pause

Any sufficiently advanced technology is indistinguishable from magic.

– Arthur C. Clarke

FP + Beweisasistent

Lean 4 ist sowohl eine funktionale Programmiersprache (ähnlich wie F#) als auch ein interaktiver Theorembeweiser.

Du kannst damit also sowohl eine App schreiben als auch mathematisch lückenlos beweisen, dass sie genau das tut, was sie soll.

Curry-Howard-Isomorphismus

In Lean sind *Sätze* (Theoreme) **Typen** und *Beweise* sind **Programme**.

Wenn du ein Programm schreibst, das einen bestimmten Typ zurückgibt, hast du gleichzeitig den Beweis für die entsprechende mathematische Aussage geliefert.

Eigenschaften I

- General-Purpose-Power
 - Lean 4 eine vollwertige Allzweck-Programmiersprache. Sie ist fast komplett in Lean selbst geschrieben ("self-hosted"), was sie extrem flexibel und performant macht.
- Kompilation nach C
 - Lean 4 kompiliert über einen Zwischenschritt direkt nach C. Das bedeutet, dass der Code sehr effizient ist. Man kann Lean also theoretisch für Systemprogrammierung nutzen, bei der absolute Korrektheit überlebenswichtig ist.

Eigenschaften II

- Metaprogrammierung
 - Eines der mächtigsten Features: Du kannst die Sprache erweitern. Wenn dir die Syntax nicht gefällt oder du eine eigene "Domain Specific Language" (DSL) brauchst, kannst du Lean so umprogrammieren, dass es deine Befehle versteht.
- Taktiken (Tactics)
 - Anstatt Beweise mühsam händisch Schritt für Schritt zu tippen, nutzt man in Lean oft Taktiken. Das sind kleine Automatismen, die dem System sagen: Löse diesen Teil durch Induktion" oder "Vereinfache diese Gleichung". Das macht das Beweisen interaktiv und fast wie ein Puzzle.

Einführung

↪ Lean Game Server

Zusammenfassung

- funktionales Domain Modeling (DDD)
- eigenschaftsbasiertes Testen (Property Based Testing)

Links

- Domain Driven Design
- Domain Modeling Made Functional
- FsCheck
- An introduction to property-based testing
- Choosing properties for property-based testing

Hausaufgabe (Erinnerung)

- exercism.io (bis 23.03)
 - ☐ Poker (Programmieraufgabe)

Termine

- ☒ 18.02 13:00 - 17:15
- ☒ 25.02 13:00 - 17:15
- ☒ 04.03 13:00 - 17:15
- ☒ 18.03 13:00 - 17:15
- ☐ 25.03 13:00 - 17:15